Time Series Analysis

```r
# EUR/USD EXCHANGE RATE


# Clear environment
rm(list=ls())

# Load libraries
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.4.3
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
library(tseries)
```

```
## Warning: package 'tseries' was built under R version 4.4.3
```

```r
library(TSA)
```

```
## Warning: package 'TSA' was built under R version 4.4.3
```

```
## Registered S3 methods overwritten by 'TSA':
##   method       from
##   fitted.Arima forecast
##   plot.Arima   forecast
```

```
##
## Attaching package: 'TSA'
```

```
## The following objects are masked from 'package:stats':
##
##     acf, arima
```

```
## The following object is masked from 'package:utils':
##
##     tar
```

```r
# Load Data
# Read CSV file
eurusd_data <- read.csv("C:\\Users\\hkatt\\Downloads\\DEXUSEU.csv", stringsAsFactors=FALSE)

# Check what we have
cat("Column names:", colnames(eurusd_data), "\n")
```

```
## Column names: observation_date DEXUSEU
```

```
cat("Number of rows:", nrow(eurusd_data), "\n")
```

```
## Number of rows: 1305
```

```
head(eurusd_data)
```

```
##   observation_date DEXUSEU
## 1       2021-01-04  1.2254
## 2       2021-01-05  1.2295
## 3       2021-01-06  1.2290
## 4       2021-01-07  1.2265
## 5       2021-01-08  1.2252
## 6       2021-01-11  1.2169
```

```
str(eurusd_data)
```

```
## 'data.frame':    1305 obs. of  2 variables:
##  $ observation_date: chr  "2021-01-04" "2021-01-05" "2021-01-06" "2021-01-07" ...
##  $ DEXUSEU         : num  1.23 1.23 1.23 1.23 1.23 ...
```

```
# Clean Data
# Rename columns if needed (FRED downloads sometimes have different names)
# Common column names: DATE, DEXUSEU or observation_date, DEXUSEU

# Check first column name and rename if needed
if(colnames(eurusd_data)[1] != "DATE") {
  colnames(eurusd_data)[1] <- "DATE"
}
if(colnames(eurusd_data)[2] != "DEXUSEU") {
  colnames(eurusd_data)[2] <- "DEXUSEU"
}

# Remove rows where DEXUSEU is "." or empty
eurusd_data <- eurusd_data[eurusd_data$DEXUSEU != ".", ]
eurusd_data <- eurusd_data[eurusd_data$DEXUSEU != "", ]
eurusd_data <- eurusd_data[!is.na(eurusd_data$DEXUSEU), ]

# Convert DEXUSEU to numeric
eurusd_data$DEXUSEU <- as.numeric(eurusd_data$DEXUSEU)

# Remove any NA values created by conversion
eurusd_data <- eurusd_data[!is.na(eurusd_data$DEXUSEU), ]

# Check cleaned data
cat("\nAfter cleaning:\n")
```

```
##
## After cleaning:
```

```r
cat("Number of rows:", nrow(eurusd_data), "\n")
```

```
## Number of rows: 1250
```

```r
head(eurusd_data)
```

```
##         DATE DEXUSEU
## 1 2021-01-04  1.2254
## 2 2021-01-05  1.2295
## 3 2021-01-06  1.2290
## 4 2021-01-07  1.2265
## 5 2021-01-08  1.2252
## 6 2021-01-11  1.2169
```

```r
# Convert Daily to Monthly
# Parse the DATE column
# Try different date formats
eurusd_data$DATE <- tryCatch({
  as.Date(eurusd_data$DATE, format="%Y-%m-%d")
}, error = function(e) {
  tryCatch({
    as.Date(eurusd_data$DATE, format="%m/%d/%Y")
  }, error = function(e) {
    as.Date(eurusd_data$DATE)
  })
})

# Check if dates parsed correctly
cat("\nDate range:", as.character(min(eurusd_data$DATE)), "to",
    as.character(max(eurusd_data$DATE)), "\n")
```

```
##
## Date range: 2021-01-04 to 2026-01-02
```

```r
# Create Year-Month column
eurusd_data$Year <- as.numeric(format(eurusd_data$DATE, "%Y"))
eurusd_data$Month <- as.numeric(format(eurusd_data$DATE, "%m"))
eurusd_data$YearMonth <- paste(eurusd_data$Year,
                               sprintf("%02d", eurusd_data$Month),
                               sep="-")

# Calculate monthly averages
monthly_data <- aggregate(DEXUSEU ~ YearMonth + Year + Month,
                          data=eurusd_data,
                          FUN=mean,
                          na.rm=TRUE)

# Sort by date
monthly_data <- monthly_data[order(monthly_data$YearMonth), ]

# Check monthly data
cat("\nMonthly data:\n")
```

```
##
## Monthly data:
```

```
head(monthly_data)
```

```
##    YearMonth Year Month  DEXUSEU
## 1    2021-01 2021     1 1.217767
## 7    2021-02 2021     2 1.209395
## 12   2021-03 2021     3 1.190161
## 17   2021-04 2021     4 1.196541
## 22   2021-05 2021     5 1.214630
## 27   2021-06 2021     6 1.204827
```

```
cat("Number of months:", nrow(monthly_data), "\n")
```

```
## Number of months: 61
```

```
# Get start date
start_year <- monthly_data$Year[1]
start_month <- monthly_data$Month[1]
cat("Start:", start_year, "-", start_month, "\n")
```

```
## Start: 2021 - 1
```

```
# Create Time Series
eurusd <- ts(monthly_data$DEXUSEU,
             start=c(start_year, start_month),
             frequency=12)

# Verify time series
cat("\nTIME SERIES INFO\n")
```

```
##
## TIME SERIES INFO
```

```
cat("Length:", length(eurusd), "\n")
```

```
## Length: 61
```

```
cat("Frequency:", frequency(eurusd), "\n")
```

```
## Frequency: 12
```

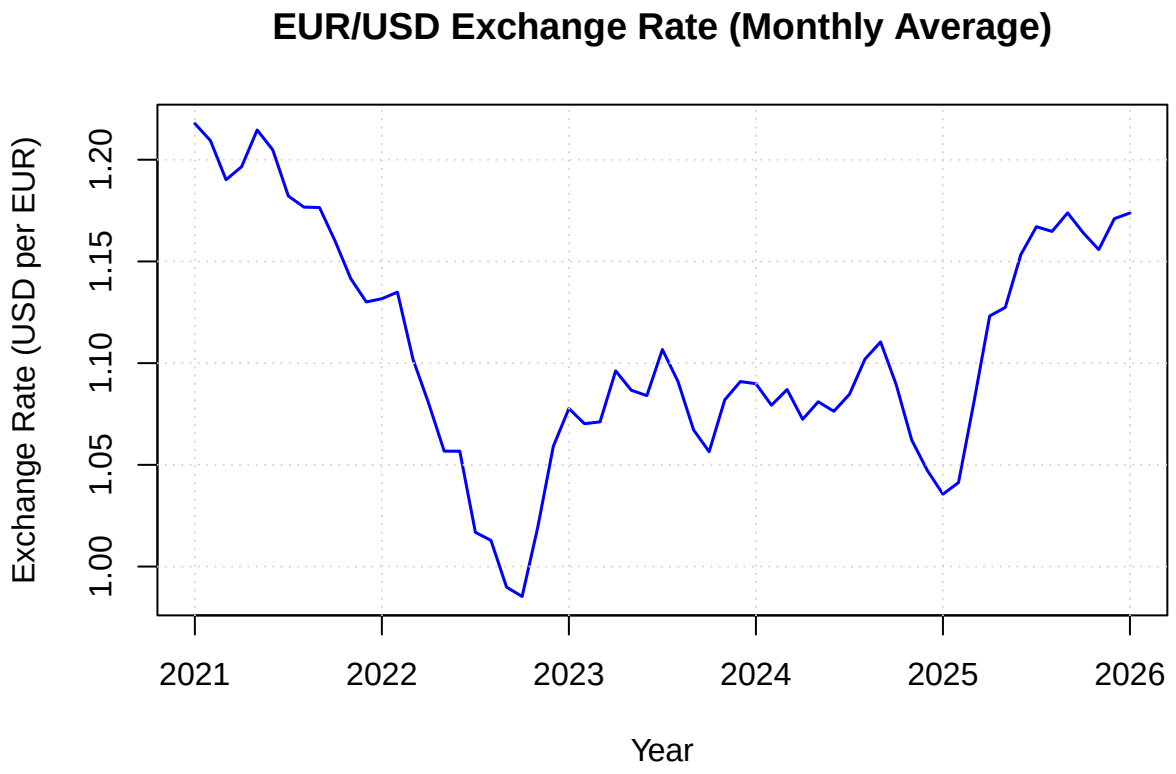
```
cat("Start:", start(eurusd), "\n")
```

```
## Start: 2021 1
```

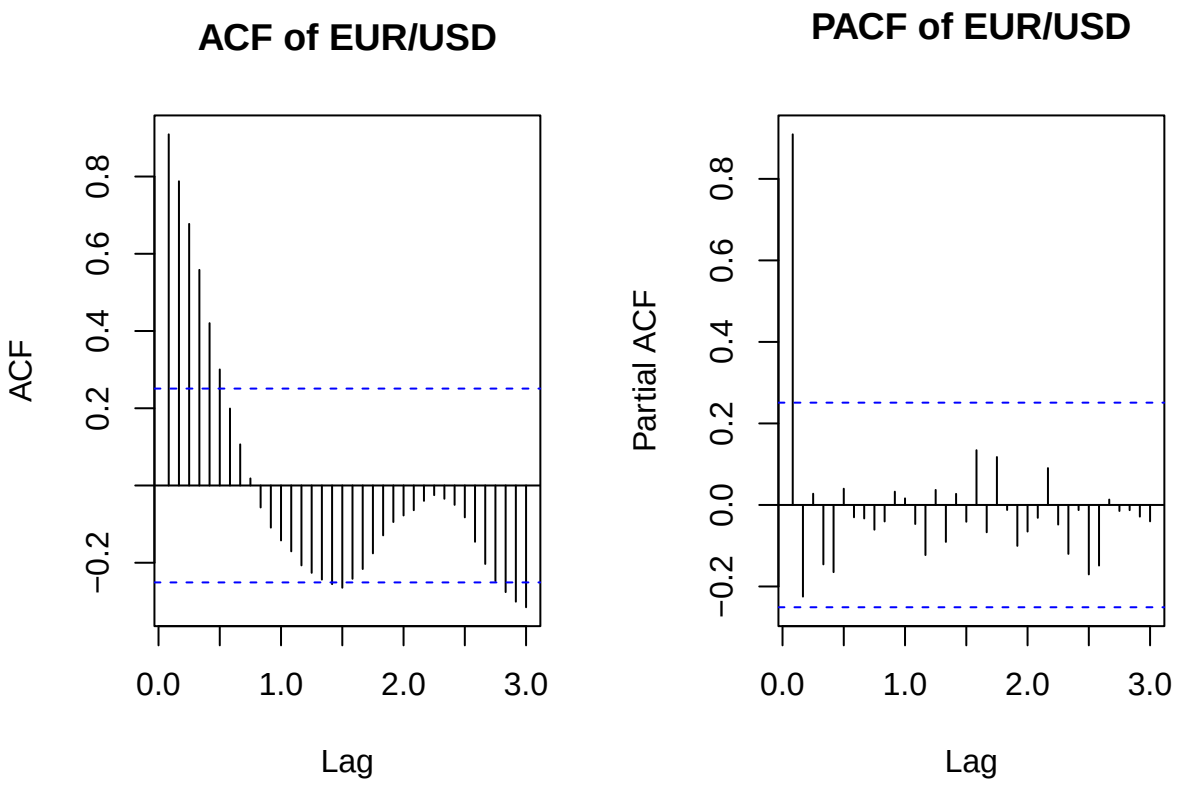
```r
cat("End:", end(eurusd), "\n")
```

```
## End: 2026 1
```

```r
# ANALYSIS BEGINS HERE

# Step 1: Plot the Data
par(mfrow=c(1,1))
plot(eurusd, main="EUR/USD Exchange Rate (Monthly Average)",
     ylab="Exchange Rate (USD per EUR)", xlab="Year",
     col="blue", lwd=1.5)
grid()
```




```r
# Step 2: Check Stationarity
par(mfrow=c(1,2))
acf(eurusd, lag.max=36, main="ACF of EUR/USD")
pacf(eurusd, lag.max=36, main="PACF of EUR/USD")
```



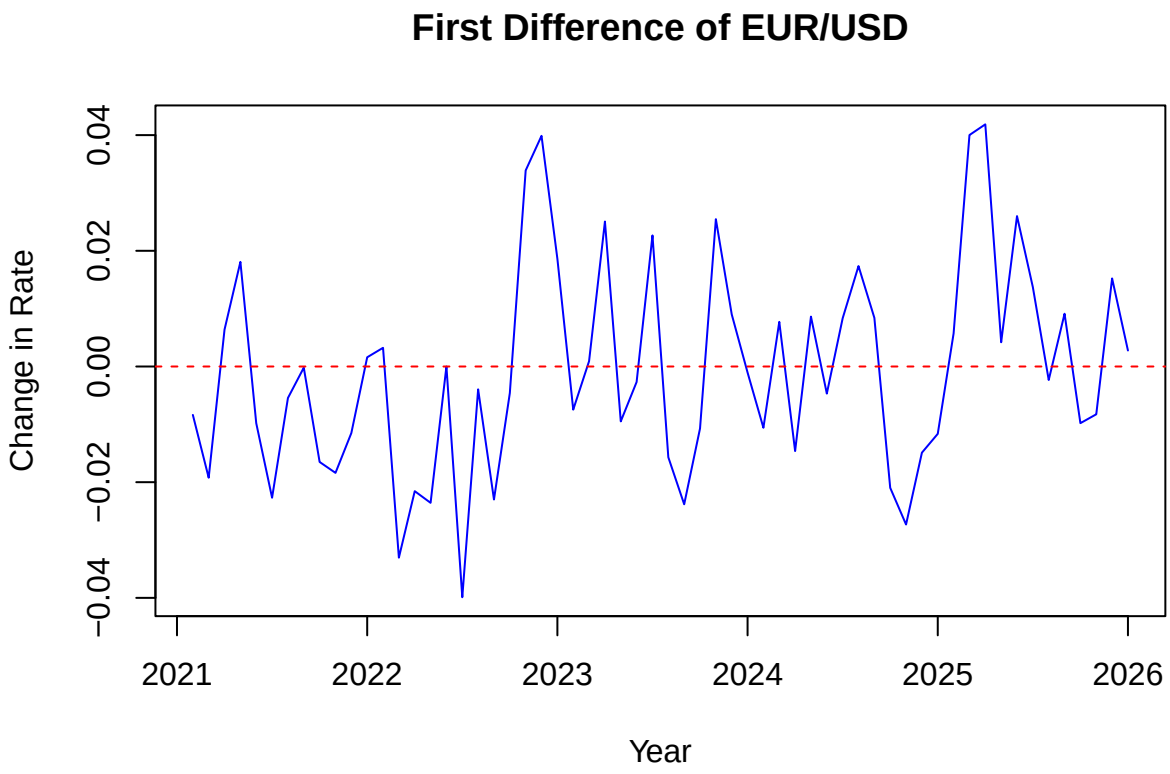

```
# ADF Test
cat("\nADF TEST (Original Series)\n")
```

```
##
## ADF TEST (Original Series)
```

```
adf_original <- adf.test(eurusd)
print(adf_original)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  eurusd
## Dickey-Fuller = -1.8467, Lag order = 3, p-value = 0.6368
## alternative hypothesis: stationary
```

```
# Step 3: First Difference
diff_eurusd <- diff(eurusd)

par(mfrow=c(1,1))
plot(diff_eurusd, main="First Difference of EUR/USD",
     ylab="Change in Rate", xlab="Year", col="blue")
abline(h=0, col="red", lty=2)
```



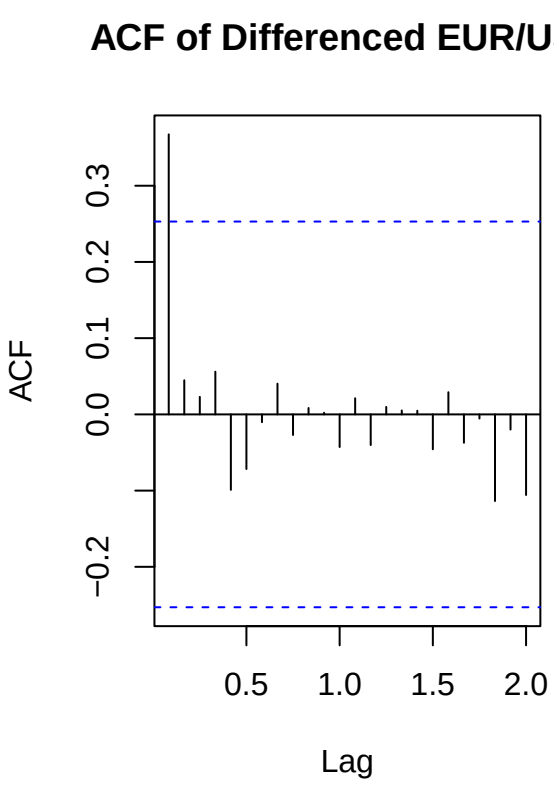

```
# ADF test on differenced series
cat("\nADF TEST (Differenced Series)\n")
```

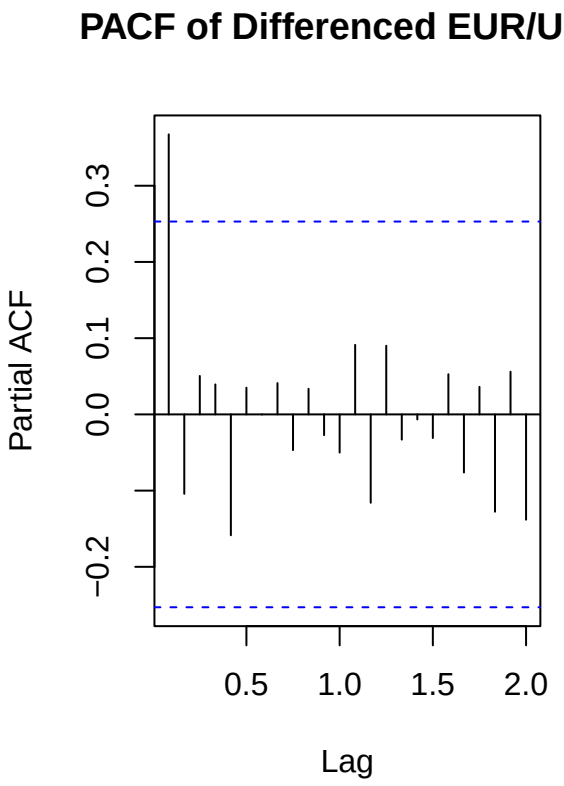
```
##
## ADF TEST (Differenced Series)
```

```
adf_diff <- adf.test(diff_eurusd)
print(adf_diff)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  diff_eurusd
## Dickey-Fuller = -3.6389, Lag order = 3, p-value = 0.03752
## alternative hypothesis: stationary
```

```
# ACF/PACF of differenced series
par(mfrow=c(1,2))
acf(diff_eurusd, lag.max=24, main="ACF of Differenced EUR/USD")
pacf(diff_eurusd, lag.max=24, main="PACF of Differenced EUR/USD")
```






```
# EACF
cat("\nEACF\n")
```

```
##
## EACF
```

```
eacf(diff_eurusd)
```

```
## AR/MA
##   0 1 2 3 4 5 6 7 8 9 10 11 12 13
## 0 x o o o o o o o o o o  o  o  o
## 1 o o o o o o o o o o o  o  o  o
## 2 x o o o o o o o o o o  o  o  o
## 3 x o o o o o o o o o o  o  o  o
## 4 o x x x o o o o o o o  o  o  o
## 5 o o o o o o o o o o o  o  o  o
## 6 o o o o o o o o o o o  o  o  o
## 7 o o o o o o o o o o o  o  o  o
```

```
# Step 4: Fit Candidate Models
cat("\nFITTING MODELS\n")
```

```
##
## FITTING MODELS
```

```r
model1 <- Arima(eurusd, order=c(0,1,0))  # Random Walk
model2 <- Arima(eurusd, order=c(1,1,0))  # ARIMA(1,1,0)
model3 <- Arima(eurusd, order=c(0,1,1))  # ARIMA(0,1,1)
model4 <- Arima(eurusd, order=c(1,1,1))  # ARIMA(1,1,1)
model5 <- Arima(eurusd, order=c(2,1,0))  # ARIMA(2,1,0)
model6 <- Arima(eurusd, order=c(0,1,2))  # ARIMA(0,1,2)
model7 <- Arima(eurusd, order=c(2,1,1))  # ARIMA(2,1,1)

# Auto ARIMA
auto_model <- auto.arima(eurusd, seasonal=FALSE, stepwise=FALSE)

# Compare AIC
cat("\nMODEL COMPARISON (AIC)\n")
```

```
##
## MODEL COMPARISON (AIC)
```

```r
model_comparison <- data.frame(
  Model = c("ARIMA(0,1,0)", "ARIMA(1,1,0)", "ARIMA(0,1,1)",
            "ARIMA(1,1,1)", "ARIMA(2,1,0)", "ARIMA(0,1,2)",
            "ARIMA(2,1,1)", "Auto ARIMA"),
  AIC = c(AIC(model1), AIC(model2), AIC(model3), AIC(model4),
          AIC(model5), AIC(model6), AIC(model7), AIC(auto_model)),
  BIC = c(BIC(model1), BIC(model2), BIC(model3), BIC(model4),
          BIC(model5), BIC(model6), BIC(model7), BIC(auto_model))
)
model_comparison <- model_comparison[order(model_comparison$AIC), ]
print(model_comparison)
```

```
##          Model       AIC       BIC
## 3 ARIMA(0,1,1) -315.9099 -311.7212
## 8   Auto ARIMA -315.9099 -311.7212
## 2 ARIMA(1,1,0) -315.5210 -311.3324
## 6 ARIMA(0,1,2) -314.2122 -307.9291
## 4 ARIMA(1,1,1) -314.1622 -307.8791
## 5 ARIMA(2,1,0) -314.1219 -307.8388
## 7 ARIMA(2,1,1) -312.1819 -303.8045
## 1 ARIMA(0,1,0) -308.8836 -306.7892
```

```r
cat("\nAuto ARIMA selected:", arimaorder(auto_model), "\n")
```

```
##
## Auto ARIMA selected: 0 1 1
```

```r
# Step 5: Select Best Model
# Find model with lowest AIC
best_model <- auto_model  # Usually auto.arima is good

cat("\nBEST MODEL SUMMARY\n")
```

```
##
## BEST MODEL SUMMARY
```

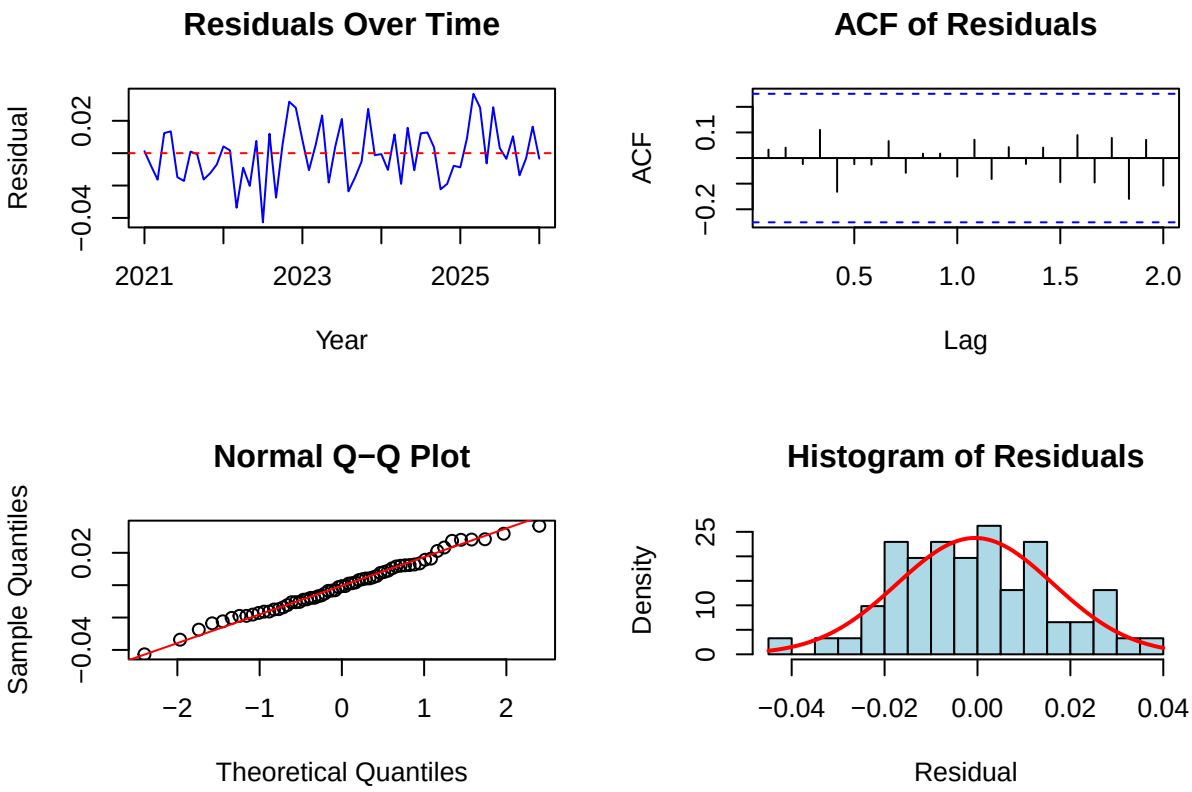
```r
summary(best_model)
```

```
## Series: eurusd
## ARIMA(0,1,1)
##
## Coefficients:
##          ma1
##       0.3732
## s.e.  0.1075
##
## sigma^2 = 0.0002872:  log likelihood = 159.95
## AIC=-315.91   AICc=-315.7   BIC=-311.72
##
## Training set error measures:
##                         ME       RMSE        MAE         MPE     MAPE      MASE
## Training set -0.0005148686 0.01666699 0.01349463 -0.05057355 1.235607 0.1975671
##                    ACF1
## Training set 0.03293305
```

```r
# Step 6: Model Diagnostics
cat("\nMODEL DIAGNOSTICS\n")
```

```
##
## MODEL DIAGNOSTICS
```

```r
# Residual plots
par(mfrow=c(2,2))

plot(residuals(best_model), main="Residuals Over Time",
     ylab="Residual", xlab="Year", col="blue")
abline(h=0, col="red", lty=2)

acf(residuals(best_model), lag.max=24, main="ACF of Residuals")

qqnorm(residuals(best_model), main="Normal Q-Q Plot")
qqline(residuals(best_model), col="red")

hist(residuals(best_model), breaks=25, main="Histogram of Residuals",
     xlab="Residual", col="lightblue", freq=FALSE)
curve(dnorm(x, mean=mean(residuals(best_model)),
            sd=sd(residuals(best_model))), add=TRUE, col="red", lwd=2)
```




```
# Ljung-Box Tests
cat("\nLjung-Box Tests:\n")
```

```
##
## Ljung-Box Tests:
```

```
for(lag in c(6, 12, 18, 24)) {
  lb <- Box.test(residuals(best_model), lag=lag, type="Ljung-Box")
  cat("Lag", lag, ": Q =", round(lb$statistic, 2),
      ", p-value =", round(lb$p.value, 4), "\n")
}
```

```
## Lag 6 : Q = 2.26 , p-value = 0.894
## Lag 12 : Q = 3.33 , p-value = 0.9927
## Lag 18 : Q = 5.43 , p-value = 0.998
## Lag 24 : Q = 11.85 , p-value = 0.9816
```

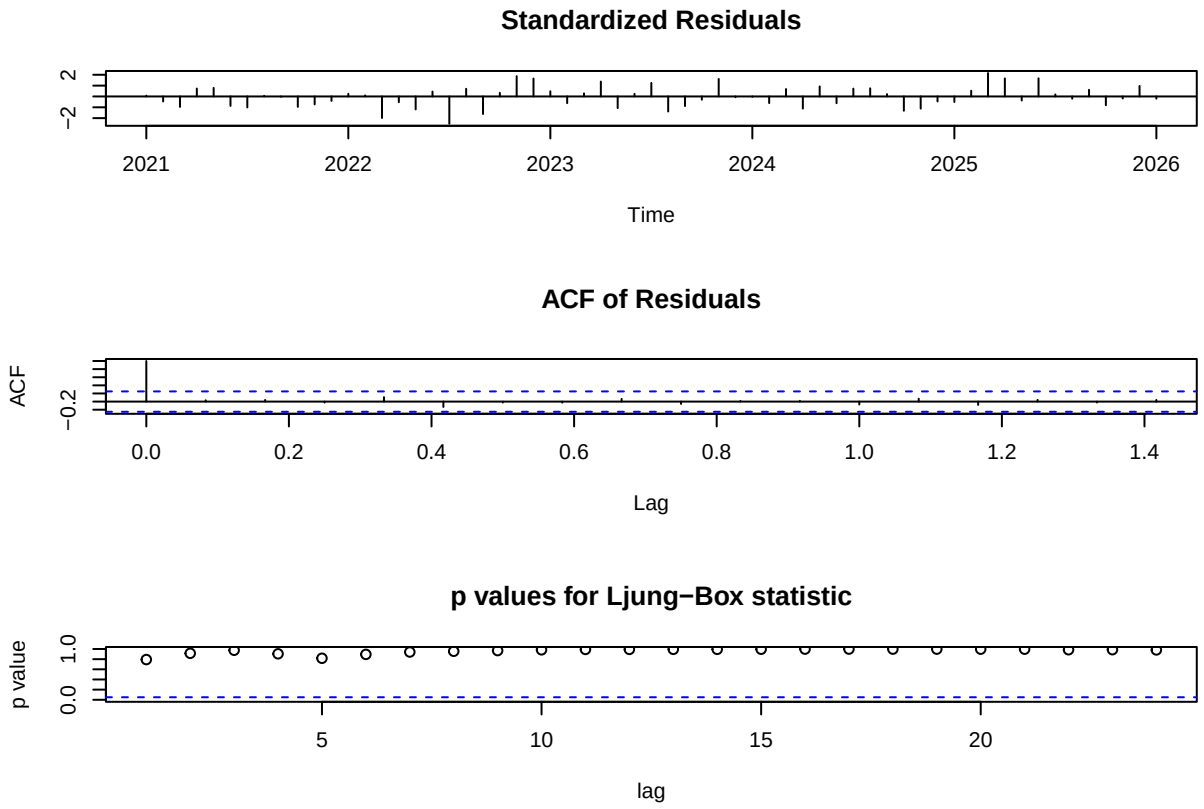
```
# Shapiro-Wilk Test
cat("\nShapiro-Wilk Test:\n")
```

```
##
## Shapiro-Wilk Test:
```

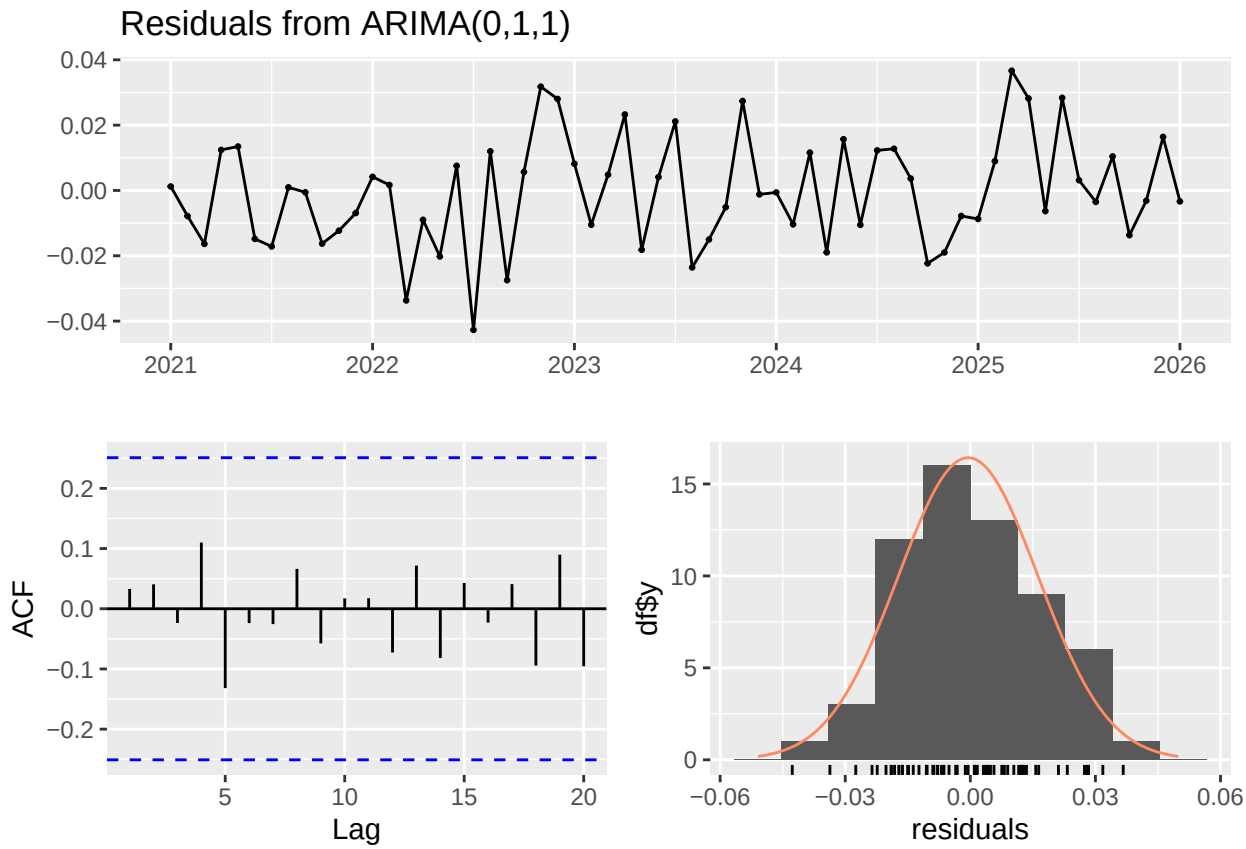
```r
sw <- shapiro.test(residuals(best_model))
cat("W =", round(sw$statistic, 4), ", p-value =", round(sw$p.value, 4), "\n")
```

```
## W = 0.9901 , p-value = 0.9029
```

```r
# tsdiag
par(mfrow=c(1,1))
tsdiag(best_model, gof.lag=24)
```




```r
# checkresiduals from forecast package
checkresiduals(best_model)
```

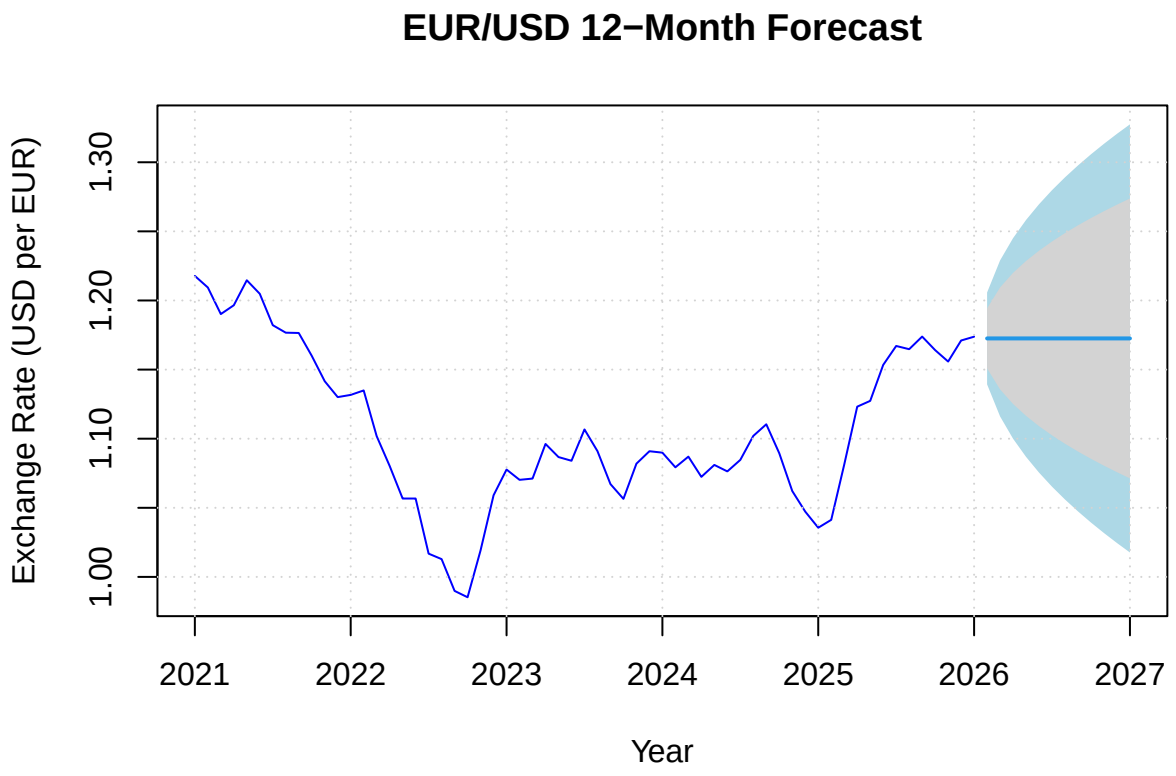



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)
## Q* = 3.3327, df = 11, p-value = 0.9856
##
## Model df: 1.   Total lags used: 12
```

```
# Step 7: Forecasting
cat("\nFORECASTING\n")
```

```
##
## FORECASTING
```

```
# Forecast next 12 months
fc <- forecast(best_model, h=12, level=c(80, 95))

# Plot forecast
par(mfrow=c(1,1))
plot(fc, main="EUR/USD 12-Month Forecast",
     xlab="Year", ylab="Exchange Rate (USD per EUR)",
     col="blue", shadecols=c("lightblue", "lightgray"))
grid()
```




```
# Print forecast
cat("\nForecast Values:\n")
```

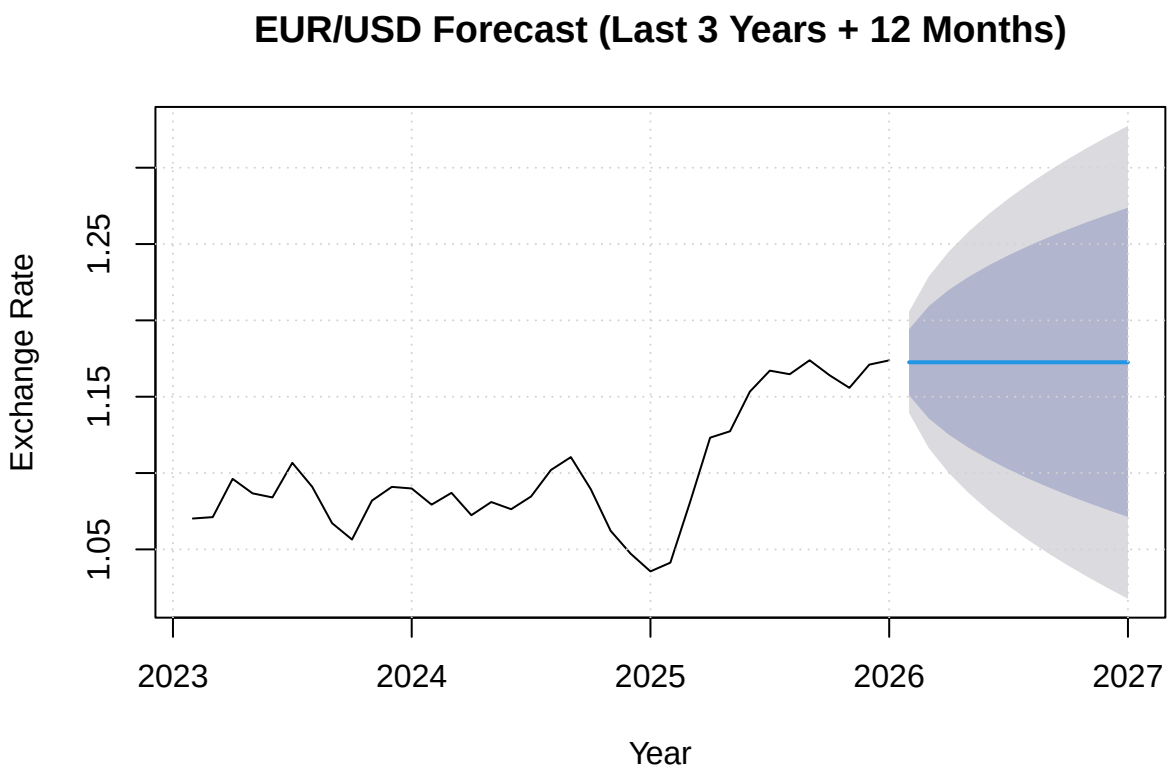
```
##
## Forecast Values:
```

```
print(fc)
```

```
##          Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## Feb 2026       1.172547 1.150829 1.194266 1.139332 1.205763
## Mar 2026       1.172547 1.135654 1.209441 1.116124 1.228971
## Apr 2026       1.172547 1.125108 1.219987 1.099994 1.245100
## May 2026       1.172547 1.116512 1.228583 1.086849 1.258246
## Jun 2026       1.172547 1.109070 1.236025 1.075467 1.269628
## Jul 2026       1.172547 1.102413 1.242682 1.065286 1.279808
## Aug 2026       1.172547 1.096336 1.248759 1.055992 1.289103
## Sep 2026       1.172547 1.090708 1.254386 1.047385 1.297709
## Oct 2026       1.172547 1.085444 1.259651 1.039334 1.305761
## Nov 2026       1.172547 1.080480 1.264615 1.031742 1.313353
## Dec 2026       1.172547 1.075770 1.269325 1.024539 1.320556
## Jan 2027       1.172547 1.071279 1.273816 1.017670 1.327424
```

```
# Zoomed plot (last 3 years + forecast)
plot(fc, include=36, main="EUR/USD Forecast (Last 3 Years + 12 Months)",
```

```
     xlab="Year", ylab="Exchange Rate")
grid()
```




```
# Summary for Report
cat("\nSUMMARY FOR REPORT \n")
```

```
##
## SUMMARY FOR REPORT
```

```
cat("Dataset: EUR/USD Exchange Rate (Monthly Average)\n")
```

```
## Dataset: EUR/USD Exchange Rate (Monthly Average)
```

```
cat("Period:", start_year, "-", start_month, "to", end(eurusd)[1], "-", end(eurusd)[2], "\n")
```

```
## Period: 2021 - 1 to 2026 - 1
```

```
cat("Observations:", length(eurusd), "\n")
```

```
## Observations: 61
```

```r
cat("Best Model: ARIMA(", paste(arimaorder(best_model), collapse=","), ")\n", sep="")
```

```
## Best Model: ARIMA(0,1,1)
```

```r
cat("AIC:", round(AIC(best_model), 2), "\n")
```

```
## AIC: -315.91
```

```r
cat("BIC:", round(BIC(best_model), 2), "\n")
```

```
## BIC: -311.72
```

```r
# US HOUSING STARTS


# Clear environment
rm(list=ls())

# Load libraries
library(forecast)
library(tseries)
library(TSA)

# Load Data
housing_data <- read.csv("C:\\Users\\hkatt\\Downloads\\HOUSTNSA.csv", stringsAsFactors=FALSE)

# Check what we have
cat("Column names:", colnames(housing_data), "\n")
```

```
## Column names: observation_date HOUSTNSA
```

```r
cat("Number of rows:", nrow(housing_data), "\n")
```

```
## Number of rows: 800
```

```r
head(housing_data)
```

```
##   observation_date HOUSTNSA
## 1       1959-01-01     96.2
## 2       1959-02-01     99.0
## 3       1959-03-01    127.7
## 4       1959-04-01    150.8
## 5       1959-05-01    152.5
## 6       1959-06-01    147.8
```

```r
str(housing_data)
```

```
## 'data.frame':    800 obs. of  2 variables:
##  $ observation_date: chr  "1959-01-01" "1959-02-01" "1959-03-01" "1959-04-01" ...
##  $ HOUSTNSA        : num  96.2 99 127.7 150.8 152.5 ...
```

```r
# Clean Data
# Rename columns if needed
if(colnames(housing_data)[1] != "DATE") {
  colnames(housing_data)[1] <- "DATE"
}
if(colnames(housing_data)[2] != "HOUSTNSA") {
  colnames(housing_data)[2] <- "HOUSTNSA"
}

# Remove rows where HOUSTNSA is "." or empty
housing_data <- housing_data[housing_data$HOUSTNSA != ".", ]
housing_data <- housing_data[housing_data$HOUSTNSA != "", ]
housing_data <- housing_data[!is.na(housing_data$HOUSTNSA), ]

# Convert to numeric
housing_data$HOUSTNSA <- as.numeric(housing_data$HOUSTNSA)

# Remove NA values
housing_data <- housing_data[!is.na(housing_data$HOUSTNSA), ]

# Check cleaned data
cat("\nAfter cleaning:\n")
```

```
##
## After cleaning:
```

```r
cat("Number of rows:", nrow(housing_data), "\n")
```

```
## Number of rows: 800
```

```r
head(housing_data)
```

```
##         DATE HOUSTNSA
## 1 1959-01-01     96.2
## 2 1959-02-01     99.0
## 3 1959-03-01    127.7
## 4 1959-04-01    150.8
## 5 1959-05-01    152.5
## 6 1959-06-01    147.8
```

```r
# Parse Dates
housing_data$DATE <- tryCatch({
  as.Date(housing_data$DATE, format="%Y-%m-%d")
}, error = function(e) {
  tryCatch({
    as.Date(housing_data$DATE, format="%m/%d/%Y")
  }, error = function(e) {
    as.Date(housing_data$DATE)
  })
})
```

```r
# Sort by date
housing_data <- housing_data[order(housing_data$DATE), ]

# Get start date
start_year <- as.numeric(format(min(housing_data$DATE), "%Y"))
start_month <- as.numeric(format(min(housing_data$DATE), "%m"))

cat("\nDate range:", as.character(min(housing_data$DATE)), "to",
    as.character(max(housing_data$DATE)), "\n")
```

```
##
## Date range: 1959-01-01 to 2025-08-01
```

```r
cat("Start:", start_year, "-", start_month, "\n")
```

```
## Start: 1959 - 1
```

```r
# Create Time Series
housing <- ts(housing_data$HOUSTNSA,
              start=c(start_year, start_month),
              frequency=12)

# Verify
cat("\nTIME SERIES INFO \n")
```

```
##
## TIME SERIES INFO
```

```r
cat("Length:", length(housing), "\n")
```

```
## Length: 800
```

```r
cat("Frequency:", frequency(housing), "\n")
```

```
## Frequency: 12
```

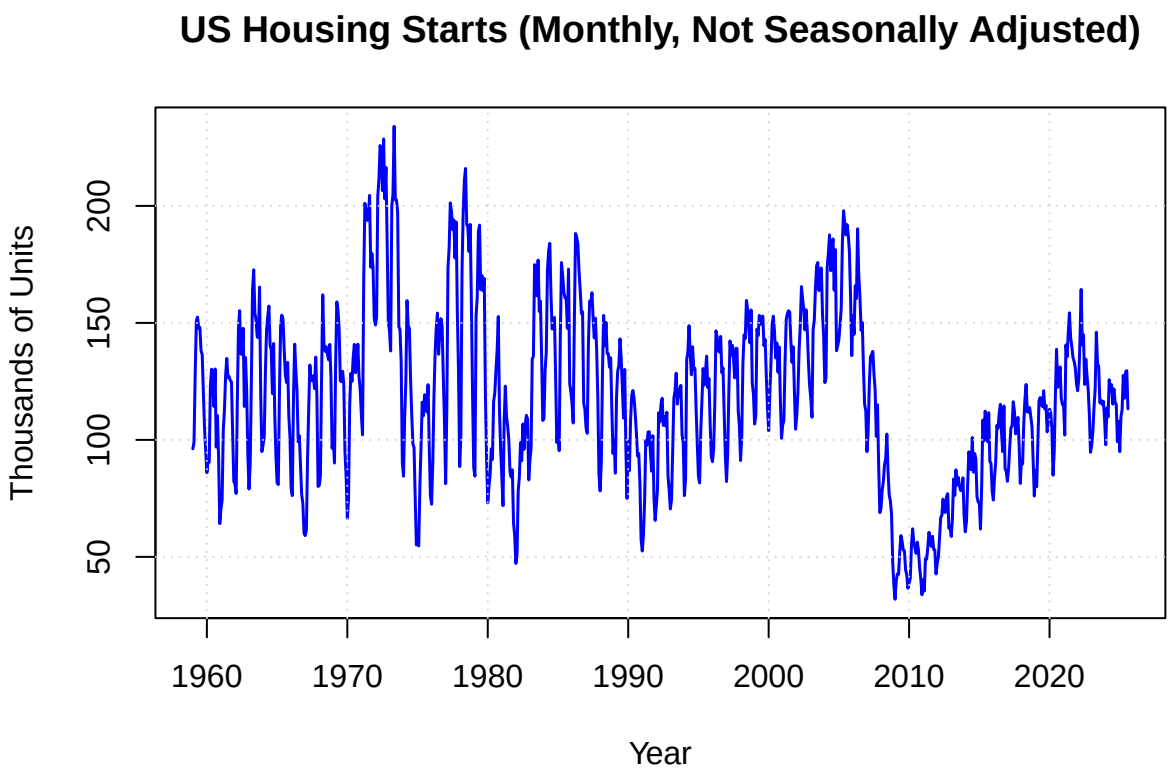
```r
cat("Start:", start(housing), "\n")
```

```
## Start: 1959 1
```

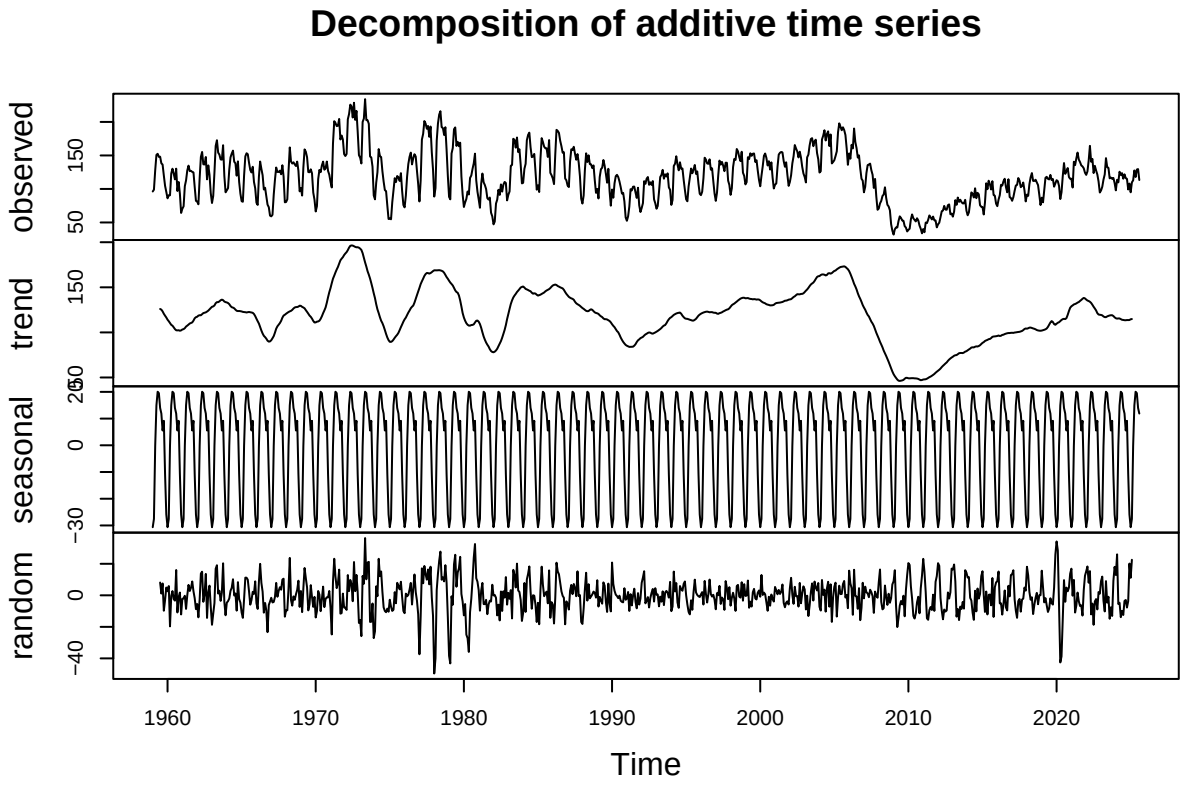
```r
cat("End:", end(housing), "\n")
```

```
## End: 2025 8
```

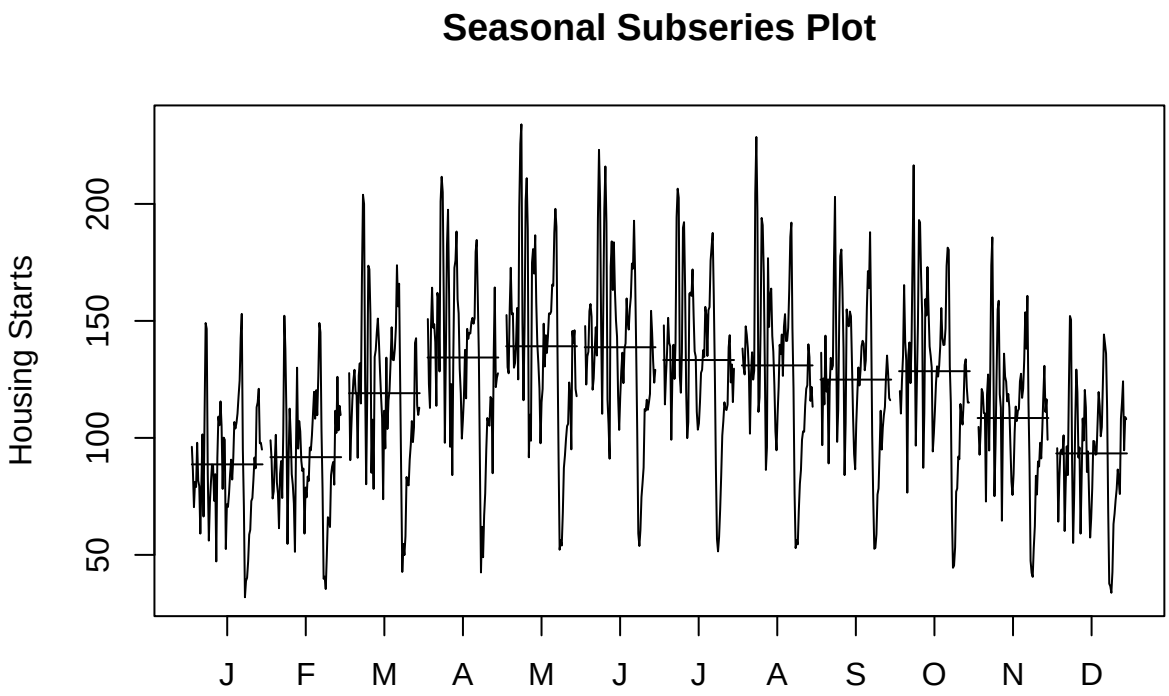
```r
# ANALYSIS BEGINS HERE


# Step 1: Plot the Data
par(mfrow=c(1,1))
plot(housing, main="US Housing Starts (Monthly, Not Seasonally Adjusted)",
     ylab="Thousands of Units", xlab="Year", col="blue", lwd=1.5)
grid()
```



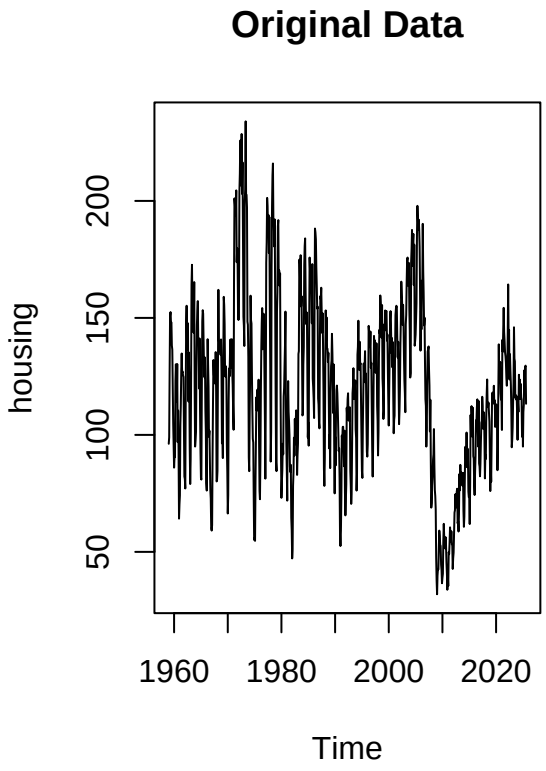

```r
# Seasonal decomposition
par(mfrow=c(1,1))
decomp <- decompose(housing)
plot(decomp)
```

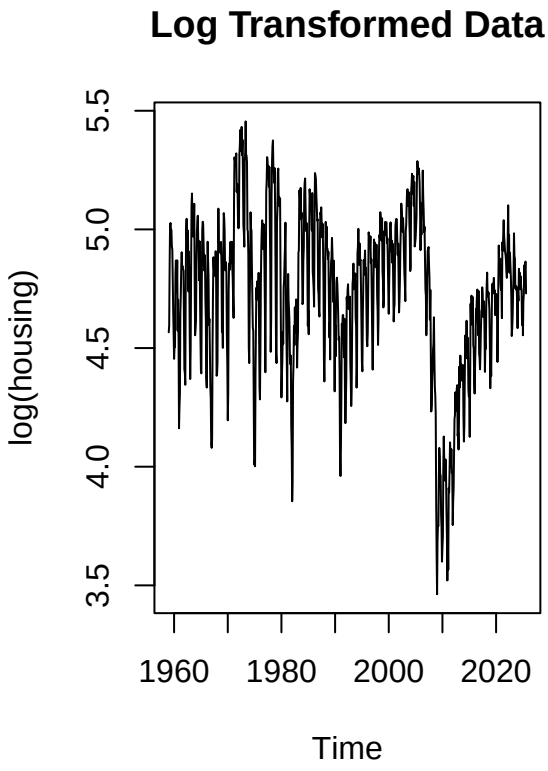



```
# Seasonal subseries plot
par(mfrow=c(1,1))
monthplot(housing, main="Seasonal Subseries Plot", ylab="Housing Starts")
```

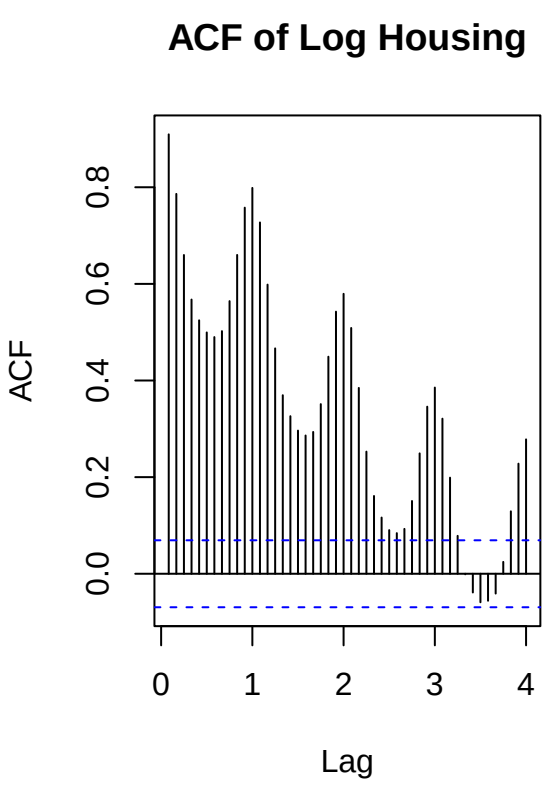


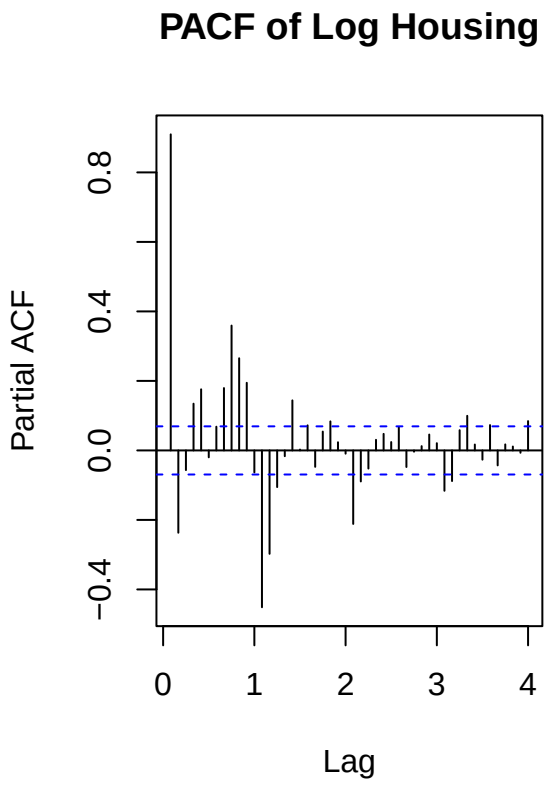

```
# Step 2: Variance Stabilization
par(mfrow=c(1,2))
plot(housing, main="Original Data")
plot(log(housing), main="Log Transformed Data")
```






```r
# Use log transformation (variance more stable)
log_housing <- log(housing)

# Step 3: Check Stationarity
par(mfrow=c(1,2))
acf(log_housing, lag.max=48, main="ACF of Log Housing")
pacf(log_housing, lag.max=48, main="PACF of Log Housing")
```





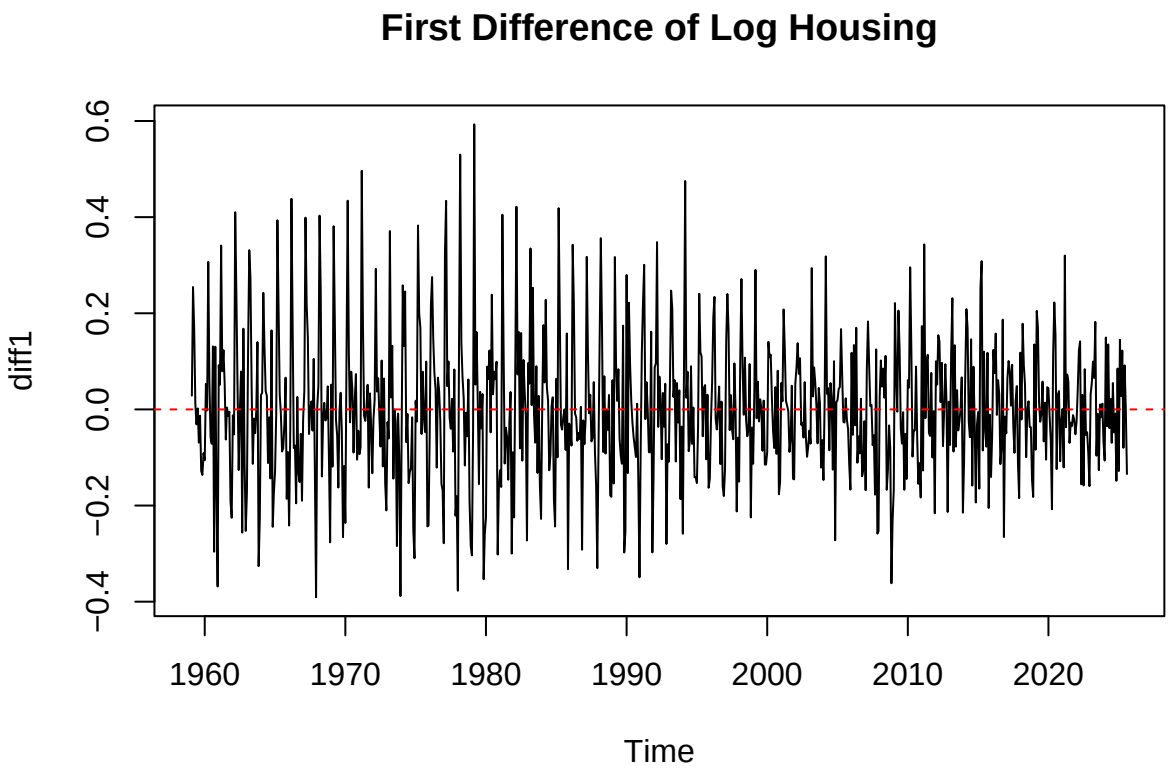

```
# ADF test
cat("\nADF TEST (Log Series)\n")
```

```
##
## ADF TEST (Log Series)
```

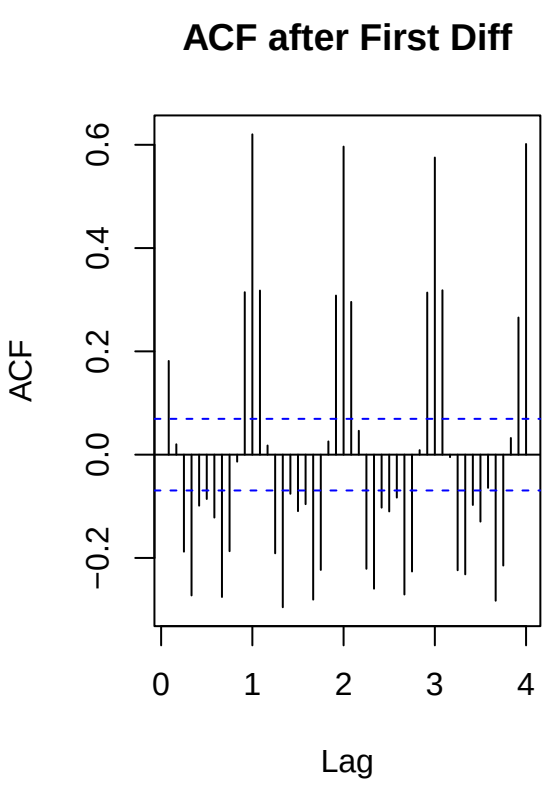
```
adf_log <- adf.test(log_housing)
print(adf_log)
```

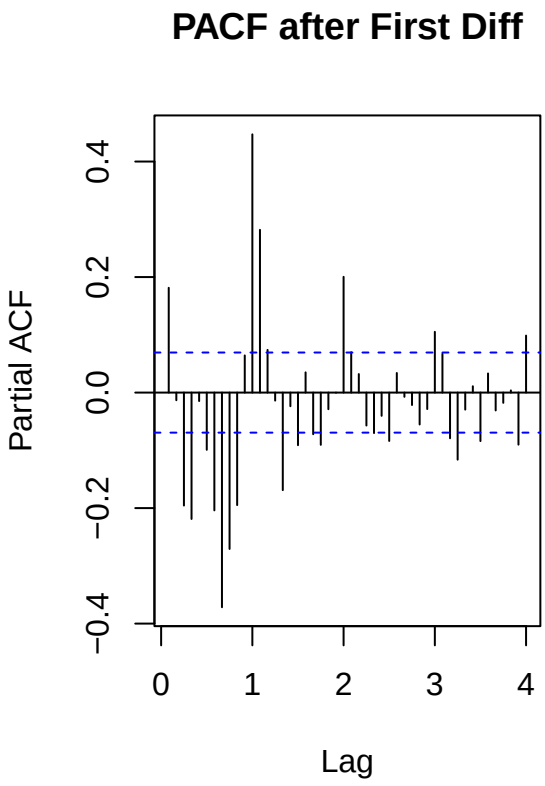
```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_housing
## Dickey-Fuller = -2.1079, Lag order = 9, p-value = 0.5326
## alternative hypothesis: stationary
```

```
# Step 4: Differencing
# First difference (d=1)
diff1 <- diff(log_housing)

par(mfrow=c(1,1))
plot(diff1, main="First Difference of Log Housing")
abline(h=0, col="red", lty=2)
```

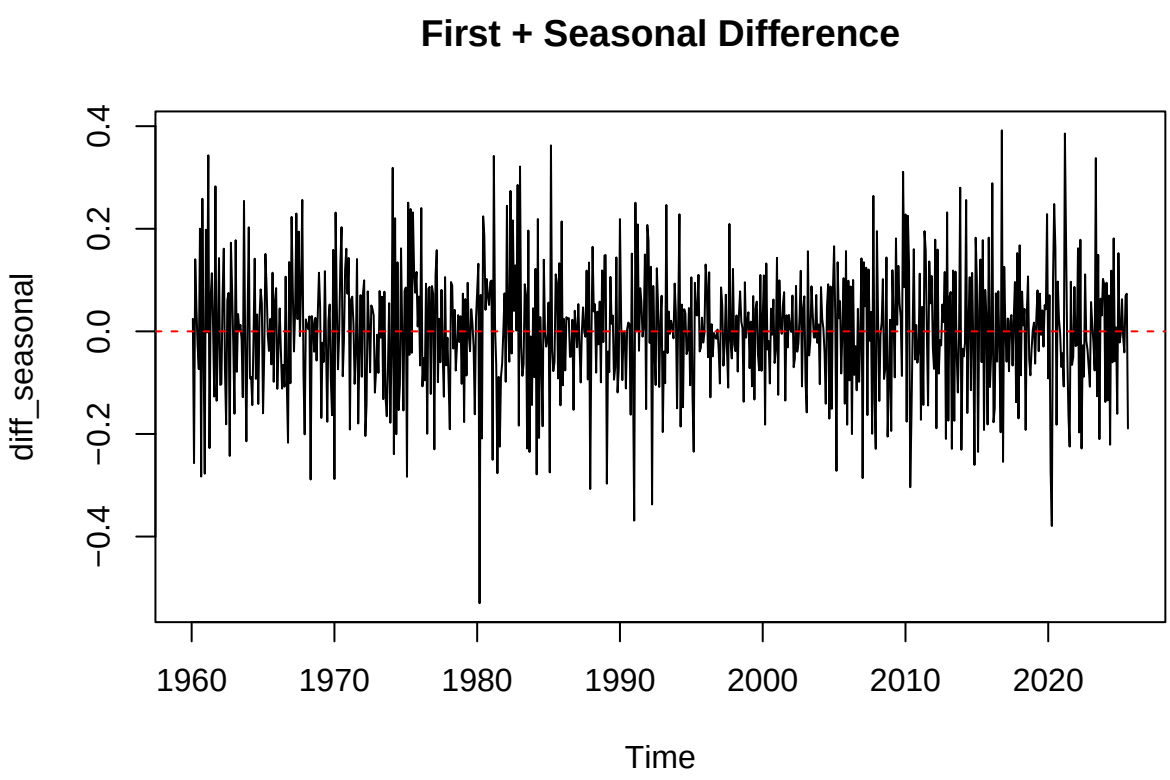



```
# Check ACF - still seasonal pattern
par(mfrow=c(1,2))
acf(diff1, lag.max=48, main="ACF after First Diff")
pacf(diff1, lag.max=48, main="PACF after First Diff")
```






```r
# Seasonal difference (D=1, lag=12)
diff_seasonal <- diff(diff1, lag=12)

par(mfrow=c(1,1))
plot(diff_seasonal, main="First + Seasonal Difference")
abline(h=0, col="red", lty=2)
```




```
# ADF test
cat("\nADF TEST (After All Differencing)\n")
```

```
##
## ADF TEST (After All Differencing)
```

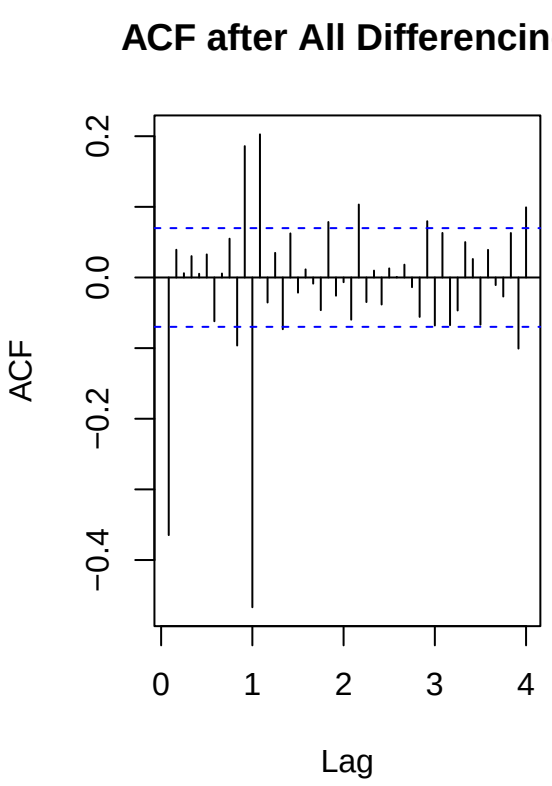
```
adf_final <- adf.test(diff_seasonal)
```

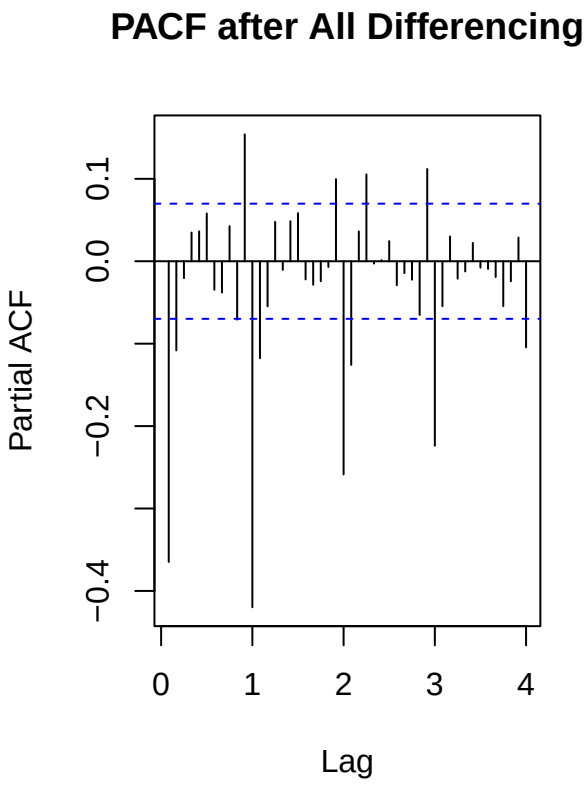
```
## Warning in adf.test(diff_seasonal): p-value smaller than printed p-value
```

```
print(adf_final)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  diff_seasonal
## Dickey-Fuller = -9.3766, Lag order = 9, p-value = 0.01
## alternative hypothesis: stationary
```

```
# ACF/PACF after all differencing
par(mfrow=c(1,2))
acf(diff_seasonal, lag.max=48, main="ACF after All Differencing")
pacf(diff_seasonal, lag.max=48, main="PACF after All Differencing")
```






```
# EACF
cat("\nEACF \n")
```

```
##
## EACF
```

```
eacf(diff_seasonal)
```

```
## AR/MA
##   0 1 2 3 4 5 6 7 8 9 10 11 12 13
## 0 x o o o o o o o o x x  x  x  o
## 1 x x o o o o o o o o o  x  x  x
## 2 x x x o o o o o o o o  x  x  x
## 3 x x x o o o o o o o o  x  x  x
## 4 x x x o x o o o o o o  x  x  x
## 5 x x x x o o o o o o o  x  x  o
## 6 x x x x o o o o o o o  x  x  o
## 7 x x x x o o o o o o o  x  x  o
```

```
# Step 5: Fit SARIMA Models
cat("\nFITTING SARIMA MODELS \n")
```

```
##
## FITTING SARIMA MODELS
```

```r
# Based on ACF/PACF patterns, fit candidate models
# ACF: spike at lag 1, spike at lag 12 -> MA(1) and SMA(1)
# PACF: decay pattern

smodel1 <- Arima(log_housing, order=c(0,1,1),
                 seasonal=list(order=c(0,1,1), period=12))

smodel2 <- Arima(log_housing, order=c(1,1,1),
                 seasonal=list(order=c(0,1,1), period=12))

smodel3 <- Arima(log_housing, order=c(0,1,1),
                 seasonal=list(order=c(1,1,1), period=12))

smodel4 <- Arima(log_housing, order=c(1,1,1),
                 seasonal=list(order=c(1,1,1), period=12))

smodel5 <- Arima(log_housing, order=c(1,1,0),
                 seasonal=list(order=c(1,1,0), period=12))

smodel6 <- Arima(log_housing, order=c(2,1,1),
                 seasonal=list(order=c(0,1,1), period=12))

smodel7 <- Arima(log_housing, order=c(0,1,2),
                 seasonal=list(order=c(0,1,1), period=12))

# Auto ARIMA with Box-Cox (Trying to fix Normality)
# lambda="auto" lets R find the best transformation to smooth outliers
auto_smodel <- auto.arima(housing, lambda = "auto", stepwise=FALSE, approximation=FALSE)

cat("\nSelected Lambda:", auto_smodel$lambda, "\n")
```

```
##
## Selected Lambda: 0.08888773
```

```r
# Compare AIC
cat("\nMODEL COMPARISON (AIC) \n")
```

```
##
## MODEL COMPARISON (AIC)
```

```r
smodel_comparison <- data.frame(
  Model = c("SARIMA(0,1,1)(0,1,1)12", "SARIMA(1,1,1)(0,1,1)12",
            "SARIMA(0,1,1)(1,1,1)12", "SARIMA(1,1,1)(1,1,1)12",
            "SARIMA(1,1,0)(1,1,0)12", "SARIMA(2,1,1)(0,1,1)12",
            "SARIMA(0,1,2)(0,1,1)12", "Auto ARIMA"),
  AIC = c(AIC(smodel1), AIC(smodel2), AIC(smodel3), AIC(smodel4),
          AIC(smodel5), AIC(smodel6), AIC(smodel7), AIC(auto_smodel)),
  BIC = c(BIC(smodel1), BIC(smodel2), BIC(smodel3), BIC(smodel4),
          BIC(smodel5), BIC(smodel6), BIC(smodel7), BIC(auto_smodel))
)
smodel_comparison <- smodel_comparison[order(smodel_comparison$AIC), ]
print(smodel_comparison)
```

```
##                     Model        AIC        BIC
## 1 SARIMA(0,1,1)(0,1,1)12 -1525.7908 -1511.7861
## 3 SARIMA(0,1,1)(1,1,1)12 -1525.6918 -1507.0189
## 7 SARIMA(0,1,2)(0,1,1)12 -1524.9772 -1506.3043
## 6 SARIMA(2,1,1)(0,1,1)12 -1524.7107 -1501.3696
## 2 SARIMA(1,1,1)(0,1,1)12 -1524.6760 -1506.0031
## 4 SARIMA(1,1,1)(1,1,1)12 -1524.6392 -1501.2980
## 5 SARIMA(1,1,0)(1,1,0)12 -1301.0546 -1287.0499
## 8             Auto ARIMA  -890.6982  -862.6812
```

```
cat("\nAuto ARIMA selected:", arimaorder(auto_smodel), "\n")
```

```
##
## Auto ARIMA selected: 3 0 0 0 1 2 12
```

```
# Step 6: Select Best Model
best_smodel <- auto_smodel  # Or choose based on lowest AIC

cat("\n BEST MODEL SUMMARY \n")
```

```
##
##  BEST MODEL SUMMARY
```

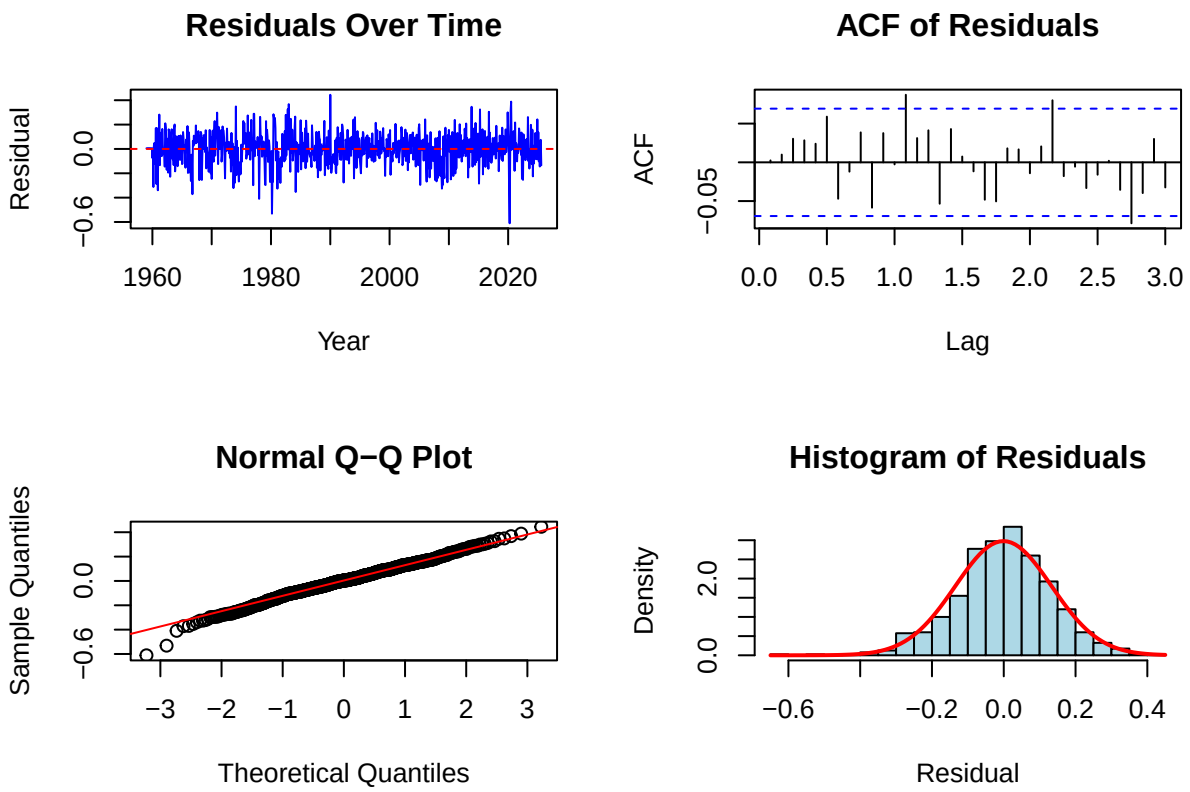
```
summary(best_smodel)
```

```
## Series: housing
## ARIMA(3,0,0)(0,1,2)[12]
## Box Cox transformation: lambda= 0.08888773
##
## Coefficients:
##          ar1     ar2     ar3     sma1     sma2
##       0.6223  0.2402  0.1013  -0.8257  -0.0522
## s.e.  0.0358  0.0412  0.0355   0.0369   0.0357
##
## sigma^2 = 0.01833:  log likelihood = 451.35
## AIC=-890.7   AICc=-890.59   BIC=-862.68
##
## Training set error measures:
##                      ME     RMSE      MAE        MPE     MAPE     MASE
## Training set 0.0720493 10.06198 7.741732 -0.4712571 6.879691 0.431578
##                       ACF1
## Training set -0.004408762
```

```
# Step 7: Model Diagnostics
cat("\nMODEL DIAGNOSTICS \n")
```

```
##
## MODEL DIAGNOSTICS
```

```r
# Residual plots
par(mfrow=c(2,2))

plot(residuals(best_smodel), main="Residuals Over Time",
     ylab="Residual", xlab="Year", col="blue")
abline(h=0, col="red", lty=2)

acf(residuals(best_smodel), lag.max=36, main="ACF of Residuals")

qqnorm(residuals(best_smodel), main="Normal Q-Q Plot")
qqline(residuals(best_smodel), col="red")

hist(residuals(best_smodel), breaks=25, main="Histogram of Residuals",
     xlab="Residual", col="lightblue", freq=FALSE)
curve(dnorm(x, mean=mean(residuals(best_smodel)),
            sd=sd(residuals(best_smodel))), add=TRUE, col="red", lwd=2)
```




```r
# Ljung-Box Tests
cat("\nLjung-Box Tests:\n")
```

```
##
## Ljung-Box Tests:
```

```r
for(lag in c(12, 24, 36)) {
  lb <- Box.test(residuals(best_smodel), lag=lag, type="Ljung-Box")
  cat("Lag", lag, ": Q =", round(lb$statistic, 2),
      ", p-value =", round(lb$p.value, 4), "\n")
}
```

```
## Lag 12 : Q = 11.81 , p-value = 0.461
## Lag 24 : Q = 28.87 , p-value = 0.225
## Lag 36 : Q = 45.15 , p-value = 0.1411
```

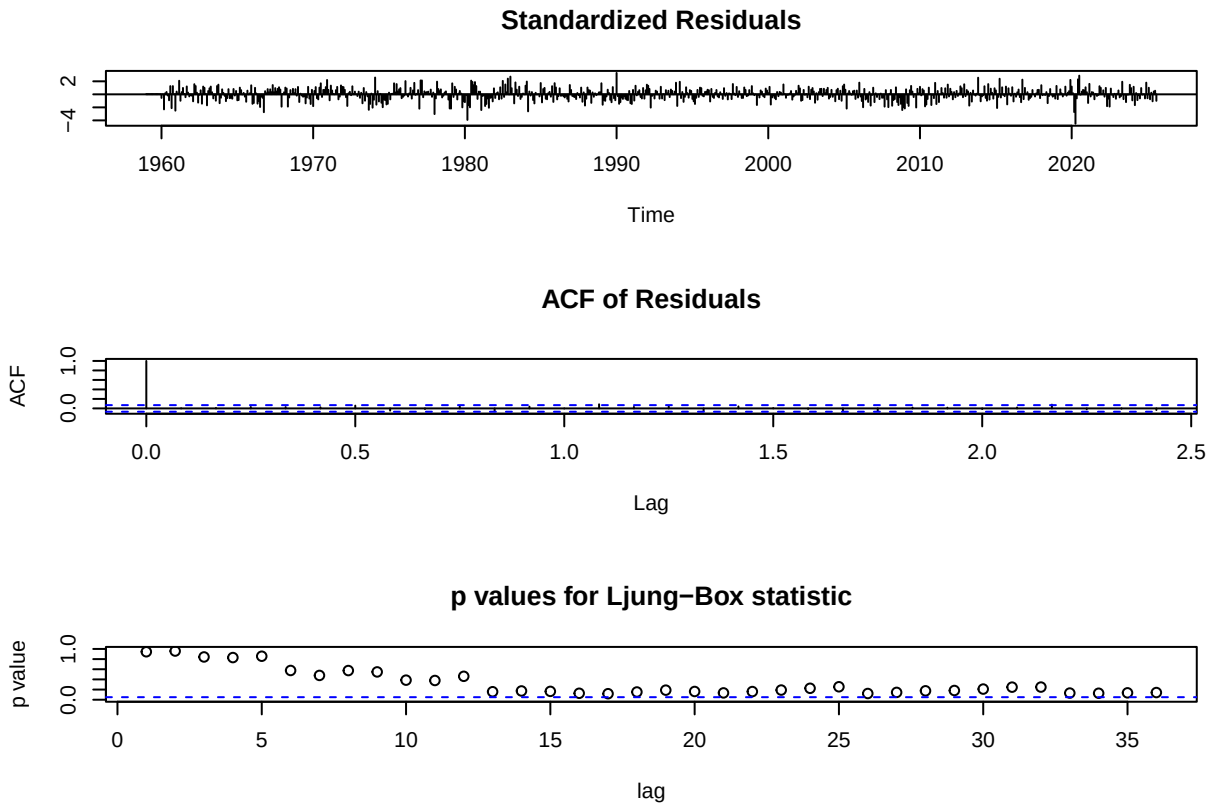
```r
# Shapiro-Wilk Test
cat("\nShapiro-Wilk Test:\n")
```

```
##
## Shapiro-Wilk Test:
```

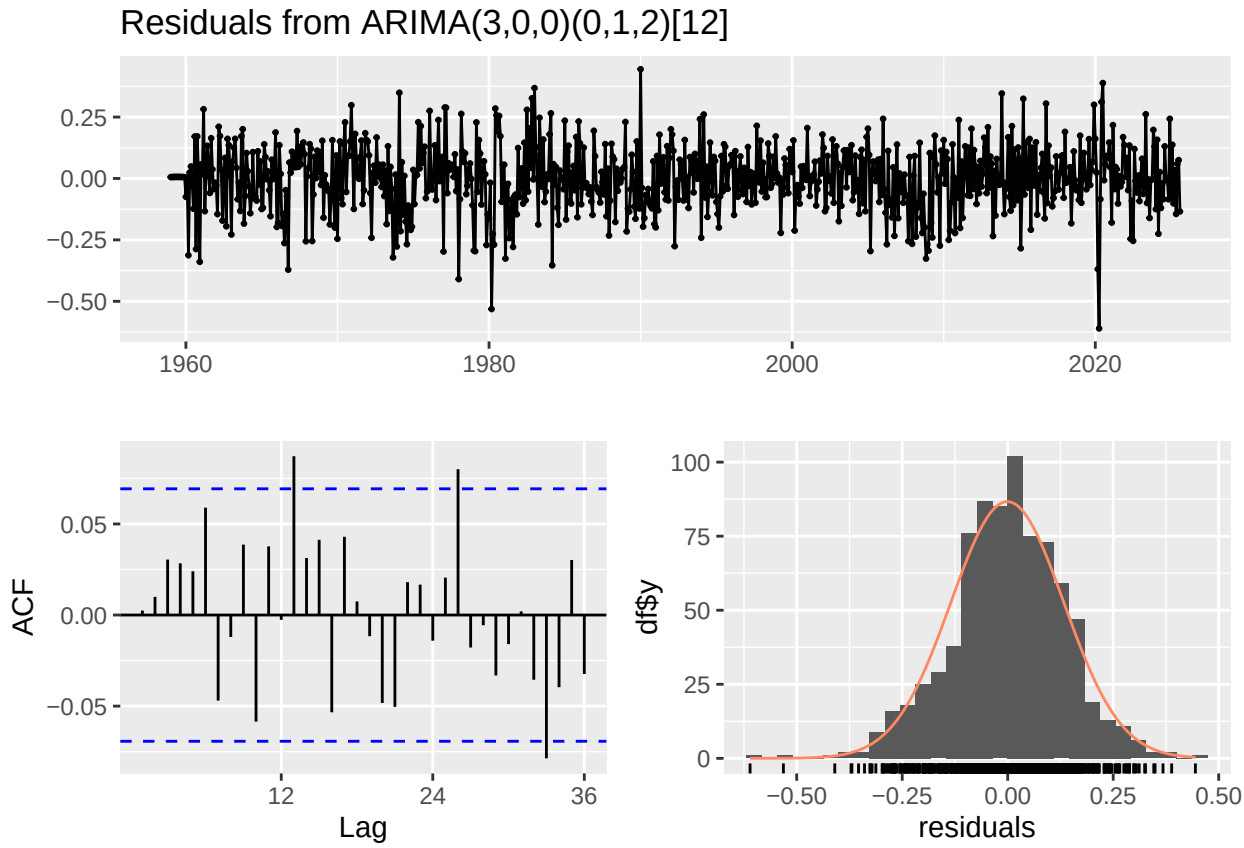
```r
sw <- shapiro.test(residuals(best_smodel))
cat("W =", round(sw$statistic, 4), ", p-value =", round(sw$p.value, 4), "\n")
```

```
## W = 0.9932 , p-value = 0.001
```

```r
# tsdiag
par(mfrow=c(1,1))
tsdiag(best_smodel, gof.lag=36)
```



```
# checkresiduals
checkresiduals(best_smodel)
```



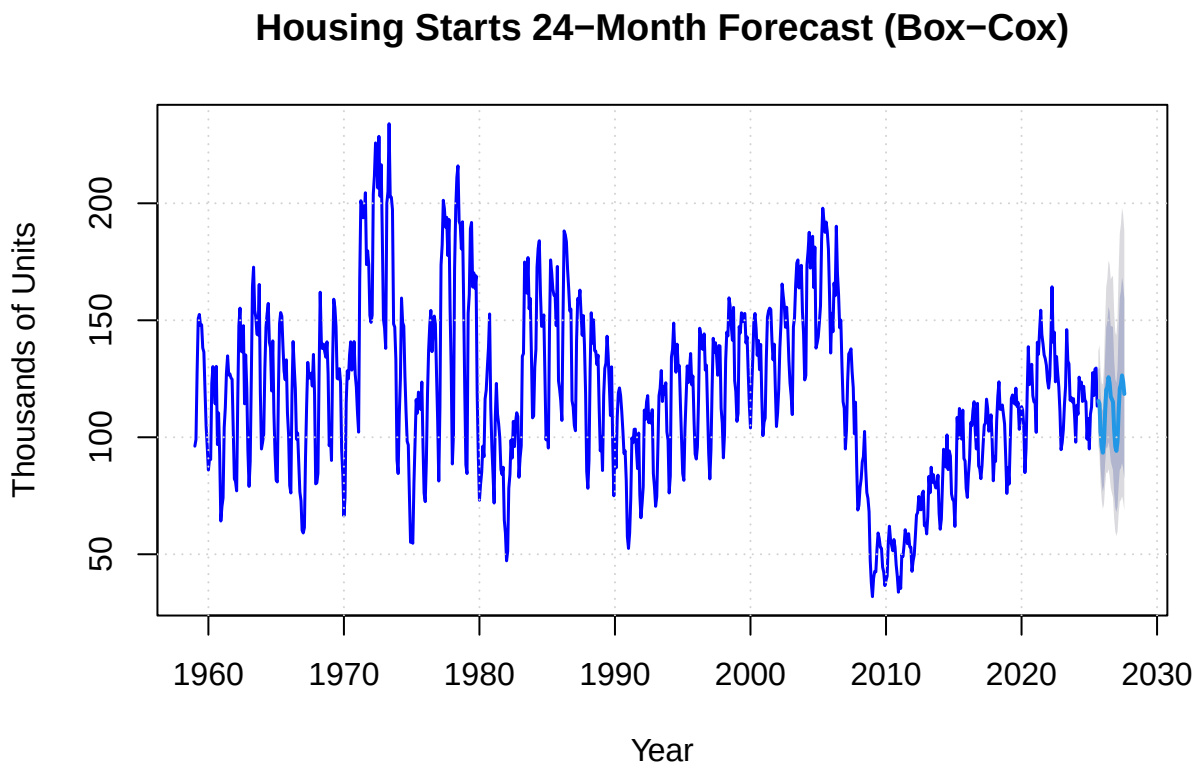

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(3,0,0)(0,1,2)[12]
## Q* = 28.871, df = 19, p-value = 0.06804
##
## Model df: 5.   Total lags used: 24
```

```
# Step 8: Forecasting
cat("\n FORECASTING \n")
```

```
##
##  FORECASTING
```

```
# Forecast next 24 months
# Note: Because we used lambda="auto" in the model, this function
# AUTOMATICALLY back-transforms the data to the original scale.
# Biasadj=TRUE ensures the mean is calculated correctly.
fc_original <- forecast(best_smodel, h=24, level=c(80, 95), biasadj=TRUE)

# Plot Forecast
```

```r
par(mfrow=c(1,1))
plot(fc_original, main="Housing Starts 24-Month Forecast (Box-Cox)",
     xlab="Year", ylab="Thousands of Units",
     xlim=c(start(housing)[1], end(housing)[1] + 3),
     col="blue", lwd=1.5)
grid()
```




```r
# Print forecast values (Already in original scale!)
cat("\nForecast Values (Original Scale - Thousands of Units):\n")
```

```
##
## Forecast Values (Original Scale - Thousands of Units):
```

```r
forecast_df <- data.frame(
  Period = as.character(time(fc_original$mean)),
  Forecast = round(as.numeric(fc_original$mean), 1),
  Lower_95 = round(as.numeric(fc_original$lower[,2]), 1),
  Upper_95 = round(as.numeric(fc_original$upper[,2]), 1)
)
print(forecast_df)
```

```
##              Period Forecast Lower_95 Upper_95
## 1  2025.66666666667    115.5     96.6    136.8
## 2           2025.75    114.3     92.4    139.4
```

```
## 3  2025.83333333333    102.1     80.0    128.0
## 4  2025.91666666667     95.9     72.9    123.2
## 5              2026     93.4     69.4    122.5
## 6  2026.08333333333     97.9     71.3    130.6
## 7  2026.16666666667    110.5     79.2    149.1
## 8           2026.25    120.3     85.1    164.2
## 9  2026.33333333333    121.5     84.6    167.9
## 10 2026.41666666667    125.7     86.4    175.5
## 11           2026.5    123.2     83.5    173.9
## 12 2026.58333333333    117.4     78.5    167.5
## 13 2026.66666666667    116.2     76.2    168.6
## 14          2026.75    115.1     74.2    168.9
## 15 2026.83333333333    103.4     65.5    154.0
## 16 2026.91666666667     96.0     59.8    144.9
## 17             2027     94.2     57.8    143.5
## 18 2027.08333333333     98.2     59.7    150.7
## 19 2027.16666666667    111.3     67.3    171.4
## 20          2027.25    120.8     72.6    187.0
## 21 2027.33333333333    122.7     73.1    191.0
## 22 2027.41666666667    126.4     74.8    198.0
## 23           2027.5    123.7     72.5    195.1
## 24 2027.58333333333    118.5     68.8    188.2
```

```
# Summary for Report
cat("\n SUMMARY FOR REPORT \n")
```

```
##
##  SUMMARY FOR REPORT
```

```
cat("Dataset: US Housing Starts (Not Seasonally Adjusted)\n")
```

```
## Dataset: US Housing Starts (Not Seasonally Adjusted)
```

```
cat("Period:", start_year, "-", start_month, "to", end(housing)[1], "-", end(housing)[2], "\n")
```

```
## Period: 1959 - 1 to 2025 - 8
```

```
cat("Observations:", length(housing), "\n")
```

```
## Observations: 800
```

```
cat("Transformation: Log\n")
```

```
## Transformation: Log
```

```
cat("Best Model: SARIMA(", paste(arimaorder(best_smodel)[1:3], collapse=","),
    ")(", paste(arimaorder(best_smodel)[4:6], collapse=","), ")12\n", sep="")
```

```
## Best Model: SARIMA(3,0,0)(0,1,2)12
```

```r
cat("AIC:", round(AIC(best_smodel), 2), "\n")
```

```
## AIC: -890.7
```

```r
cat("BIC:", round(BIC(best_smodel), 2), "\n")
```

```
## BIC: -862.68
```